

Creature Class Application System — Technical Design

Document Version: 1.0

Date: 2026-05-25

Companion: `npc_classes_implementation_plan.md`, `npc_class_definitions.md`

Table of Contents

- [1. System Overview](#)
 - [2. Core Data Model Extensions](#)
 - [3. Hit Dice Calculation](#)
 - [4. Base Attack Bonus Calculation](#)
 - [5. Saving Throw Calculation](#)
 - [6. Hit Point Calculation](#)
 - [7. Skill Point Allocation](#)
 - [8. Feat Progression](#)
 - [9. Ability Score Increases](#)
 - [10. Stat Array Application](#)
 - [11. CR Calculation Formulas](#)
 - [12. ECL & Level Adjustment](#)
 - [13. Equipment Assignment](#)
 - [14. ClassLevelApplier — Core Engine](#)
 - [15. Code Structure & Class Hierarchy](#)
-

1. System Overview

The Creature Class Application System takes any base creature (from `NPCDefinition`) and applies one or more class levels to it, producing a fully recalculated `CharacterStats` with correct HD, BAB, saves, HP, skills, feats, ability scores, CR, and equipment.

1.1 Pipeline Flow



2. Core Data Model Extensions

2.1 New Fields on CharacterStats

```

public class CharacterStats
{
    // === EXISTING FIELDS (unchanged) ===
    public string CharacterName;
    public int Level;
    public List<ClassLevelEntry> ClassLevels;
    public string CharacterClass;
    public string ChallengeRating;
    public string CreatureType;
    public bool UseCreatureTypeProgression;
    public int NaturalArmorBonus;
    public int BaseSpeed;
    // ...

    // === NEW FIELDS FOR CREATURE CLASS APPLICATION ===

    /// <summary>
    /// Number of racial Hit Dice from monster type (e.g., Ogre = 4).
    /// 0 for standard humanoid PCs/NPCs with no racial HD.
    /// </summary>
    public int RacialHitDice;

    /// <summary>
    /// Level Adjustment for ECL calculation (e.g., Drow = +2).
    /// </summary>
    public int LevelAdjustment;

    /// <summary>
    /// Comma-separated class names considered "associated" for CR calculation.
    /// e.g., "Fighter,Barbarian" for an Ogre.
    /// </summary>
    public string AssociatedClassHint = "";

    /// <summary>
    /// Which stat array was applied: Elite, Nonelite, Basic, or Custom.
    /// </summary>
    public StatArrayType AppliedStatArray = StatArrayType.Custom;

    /// <summary>
    /// Total Hit Dice = RacialHitDice + sum of all ClassLevels.
    /// </summary>
    public int TotalHitDice
    {
        get
        {
            int classHD = 0;
            for (int i = 0; i < ClassLevels.Count; i++)
                classHD += ClassLevels[i].Level;
            return RacialHitDice + classHD;
        }
    }

    /// <summary>
    /// Effective Character Level = TotalHitDice + LevelAdjustment.
    /// Used for XP progression and encounter balancing for PC-role monsters.
    /// </summary>
    public int EffectiveCharacterLevel => TotalHitDice + LevelAdjustment;
}

public enum StatArrayType
{
    Custom,          // Manual/original scores

```

```

Elite,      // 15, 14, 13, 12, 10, 8
Nonelite,   // 13, 12, 11, 10, 9, 8
Basic       // 11, 11, 11, 10, 10, 10
}

```

2.2 NPCDefinition Extensions

```

// Add to NPCDefinition struct/class
public class NPCDefinition
{
    // ... existing fields ...

    /// <summary>Racial Hit Dice count from Monster Manual entry.</summary>
    public int RacialHitDice;

    /// <summary>Level Adjustment for the race (e.g., +2 for Drow).</summary>
    public int LevelAdjustment;

    /// <summary>
    /// Classes considered "associated" for this creature type.
    /// Associated classes add +1 CR per level; nonassociated add +½.
    /// </summary>
    public string[] AssociatedClasses;

    /// <summary>
    /// Whether this creature supports class level advancement.
    /// "ByCharacterClass" / "ByHD" / "None"
    /// </summary>
    public string AdvancementType = "ByCharacterClass";

    /// <summary>Base ability scores BEFORE racial modifiers (for array
    replacement).</summary>
    public int[] BaseAbilityScores; // [STR, DEX, CON, INT, WIS, CHA]

    /// <summary>Racial ability score modifiers (e.g., Ogre: +10 STR, -2 DEX, +4 CON,
    -4 INT, -4 CHA).</summary>
    public int[] RacialAbilityModifiers; // [STR, DEX, CON, INT, WIS, CHA]
}

```

3. Hit Dice Calculation

3.1 Formula

Total HD = Racial HD (from creature **type**) + Class HD (from each **class** level entry)

Each source of HD uses its own die size:

- **Racial HD:** Die size from `CreatureTypeProgression` (e.g., Giant = d8, Dragon = d12)
- **Class HD:** Die size from `ICharacterClass.HitDie`

3.2 Implementation

```
public static class HitDiceCalculator
{
    /// <summary>
    /// Calculate total Hit Dice count from racial HD and class levels.
    /// </summary>
    public static int GetTotalHD(int racialHD, List<ClassLevelEntry> classLevels)
    {
        int total = racialHD;
        for (int i = 0; i < classLevels.Count; i++)
            total += classLevels[i].Level;
        return total;
    }

    /// <summary>
    /// Get a display string like "4d8+8 + 2d12+8" (Ogre 4 HD + Barbarian 2).
    /// </summary>
    public static string GetHDString(int racialHD, int racialDie,
        List<ClassLevelEntry> classLevels, int conMod)
    {
        var parts = new List<string>();

        if (racialHD > 0)
        {
            int racialConHP = racialHD * conMod;
            string sign = racialConHP >= 0 ? "+" : "";
            parts.Add($"{racialHD}d{racialDie}{sign}{racialConHP}");
        }

        for (int i = 0; i < classLevels.Count; i++)
        {
            ICharacterClass cls = ClassRegistry.GetClass(classLevels[i].ClassName);
            if (cls == null) continue;
            int lvl = classLevels[i].Level;
            int classConHP = lvl * conMod;
            string sign = classConHP >= 0 ? "+" : "";
            parts.Add($"{lvl}d{cls.HitDie}{sign}{classConHP}");
        }

        return string.Join(" + ", parts);
    }
}
```

4. Base Attack Bonus Calculation

4.1 Formula

Total BAB = BAB(racial HD, creature BAB progression) + BAB(class levels, class BAB progression)

Each component is calculated independently using the appropriate progression track, then summed.

4.2 BAB Progression Reference

Progression	Formula	At Level 3	Example Classes
Good (+1/level)	level	3	Fighter, Barbarian, Ranger, Paladin, Warrior
Medium (+ $\frac{3}{4}$ /level)	$\text{floor}(\text{level} \times \frac{3}{4})$	2	Cleric, Druid, Rogue, Bard, Monk, Aristocrat, Expert
Poor (+ $\frac{1}{2}$ /level)	$\text{floor}(\text{level} / 2)$	1	Wizard, Sorcerer, Adept, Commoner

Creature type BAB progressions (from `CreatureTypeProgressionDatabase`):

- **Good:** Dragon, Magical Beast, Monstrous Humanoid, Outsider
- **Medium:** Aberration, Animal, Construct, Elemental, Giant, Humanoid, Ooze, Plant, Undead, Vermin
- **Poor:** Fey

4.3 Implementation

```

public static class BABCalculator
{
    /// <summary>
    /// Calculate total BAB from racial HD + all class levels.
    /// </summary>
    public static int CalculateTotalBAB(
        int racialHD,
        BABProgression racialProgression,
        List<ClassLevelEntry> classLevels)
    {
        int totalBAB = 0;

        // Racial BAB
        if (racialHD > 0)
            totalBAB += ProgressionCalculator.CalculateBAB(racialProgression, racial-
HD);

        // Class BAB
        for (int i = 0; i < classLevels.Count; i++)
        {
            ICharacterClass cls = ClassRegistry.GetClass(classLevels[i].ClassName);
            if (cls == null) continue;

            BABProgression classBAB = GetBABProgression(cls);
            totalBAB += ProgressionCalculator.CalculateBAB(classBAB, class-
Levels[i].Level);
        }

        return totalBAB;
    }

    /// <summary>
    /// Derive BAB progression from ICharacterClass.BABAtLevel3.
    /// </summary>
    public static BABProgression GetBABProgression(ICharacterClass cls)
    {
        switch (cls.BABAtLevel3)
        {
            case 3: return BABProgression.Good;
            case 2: return BABProgression.Medium;
            case 1: return BABProgression.Poor;
            default: return BABProgression.Medium;
        }
    }
}

```


4.4 Iterative Attack Calculation

Total BAB	Attacks
+1 to +5	Single attack at full BAB
+6 to +10	+6/+1 (two attacks)
+11 to +15	+11/+6/+1 (three attacks)
+16 to +20	+16/+11/+6/+1 (four attacks)

```
public static string GetIterativeAttacks(int totalBAB)
{
    var attacks = new List<string>();
    for (int bab = totalBAB; bab > 0; bab -= 5)
    {
        attacks.Add($"+{bab}");
    }
    return string.Join("/", attacks);
}
```

5. Saving Throw Calculation

5.1 Formula

Total Save = Racial Save Bonus + Class Save Bonus + Ability Modifier + Misc

Racial and class save bonuses stack additively. Each is calculated independently.

5.2 Implementation

```

public static class SaveCalculator
{
    /// <summary>
    /// Calculate combined base save bonus from racial HD + all class levels.
    /// </summary>
    public static int CalculateBaseSave(
        int racialHD,
        SaveProgression racialProgression,
        List<ClassLevelEntry> classLevels,
        Func<ICharacterClass, bool> isGoodSave)
    {
        int total = 0;

        // Racial save
        if (racialHD > 0)
            total += ProgressionCalculator.CalculateSave(racialProgression, racialHD);

        // Class saves
        for (int i = 0; i < classLevels.Count; i++)
        {
            ICharacterClass cls = ClassRegistry.GetClass(classLevels[i].ClassName);
            if (cls == null) continue;

            SaveProgression prog = isGoodSave(cls)
                ? SaveProgression.Good
                : SaveProgression.Poor;
            total += ProgressionCalculator.CalculateSave(prog, classLevels[i].Level);
        }

        return total;
    }

    // Convenience overloads:
    public static int Fort(int racialHD, SaveProgression racialFort, List<Class-
LevelEntry> cls)
        => CalculateBaseSave(racialHD, racialFort, cls, c => c.GoodFortitude);

    public static int Ref(int racialHD, SaveProgression racialRef, List<Class-
LevelEntry> cls)
        => CalculateBaseSave(racialHD, racialRef, cls, c => c.GoodReflex);

    public static int Will(int racialHD, SaveProgression racialWill, List<Class-
LevelEntry> cls)
        => CalculateBaseSave(racialHD, racialWill, cls, c => c.GoodWill);
}

```

5.3 Save Progression Reference

Save Level	Good	Poor
1	+2	+0
2	+3	+0
3	+3	+1
4	+4	+1
5	+4	+1
6	+5	+2
7	+5	+2
8	+6	+2
9	+6	+3
10	+7	+3
12	+8	+4
15	+9	+5
20	+12	+6

6. Hit Point Calculation

6.1 Formula

Total HP = Racial HP + Class HP

Racial HP:

- 1st racial HD: max die value + Con modifier
- Remaining racial HD: average roll + Con modifier per die
- Average roll = (die_size + 1) / 2 (integer division)

Class HP:

- 1st class HD (if no racial HD): max die value + Con modifier
- All other class HD: average roll + Con modifier per die
- Minimum 1 HP per die

6.2 Implementation

```

public static class HPCalculator
{
    /// <summary>
    /// Calculate total HP for a creature with racial HD + class levels.
    /// </summary>
    public static int Calculate(
        int racialHD, int racialDieSize,
        List<ClassLevelEntry> classLevels,
        int conModifier)
    {
        int totalHP = 0;
        bool firstDieMaxed = false;

        // Racial HP
        if (racialHD > 0)
        {
            // 1st HD is maximum
            totalHP += racialDieSize + conModifier;
            firstDieMaxed = true;

            // Remaining racial HD at average
            int avgRoll = (racialDieSize + 1) / 2;
            for (int i = 1; i < racialHD; i++)
                totalHP += Mathf.Max(1, avgRoll + conModifier);
        }

        // Class HP
        for (int i = 0; i < classLevels.Count; i++)
        {
            ICharacterClass cls = ClassRegistry.GetClass(classLevels[i].ClassName);
            if (cls == null) continue;

            int classDie = cls.HitDie;
            int avgRoll = (classDie + 1) / 2;

            for (int j = 0; j < classLevels[i].Level; j++)
            {
                if (!firstDieMaxed)
                {
                    // First HD ever is maxed
                    totalHP += classDie + conModifier;
                    firstDieMaxed = true;
                }
                else
                {
                    totalHP += Mathf.Max(1, avgRoll + conModifier);
                }
            }
        }

        return Mathf.Max(1, totalHP);
    }
}

```

6.3 Example: Ogre Barbarian 2

```
Ogre: 4d8 (Giant HD)
  1st HD: 8 + 4 (Con +4) = 12
  HD 2-4: avg(8) = 4, so 3 × (4+4) = 24
  Racial HP = 36

Barbarian 2: 2d12
  HD 1-2: avg(12) = 6, so 2 × (6+4) = 20
  Class HP = 20

Total HP = 36 + 20 = 56
```

7. Skill Point Allocation

7.1 Rules

- **Skill points from class levels:** $(\text{SkillPointsPerLevel} + \text{IntMod}) \times \text{classLevel}$
- **1st character level gets ×4 multiplier** (only if it's the creature's first class level AND the creature has 0 racial HD)
- **Max ranks:** Total HD + 3 for class skills; half that for cross-class
- **Monster's listed skills** are always treated as class skills in addition to the class's own skills
- **Minimum 1 skill point per level** (even with negative Int modifier)

7.2 Implementation

```

public static class CreatureSkillAllocator
{
    /// <summary>
    /// Calculate total skill points gained from adding class levels to a creature.
    /// </summary>
    public static int CalculateClassSkillPoints(
        int classSkillPointsPerLevel,
        int classLevel,
        int intModifier,
        bool isFirstClassLevel,
        int racialHD)
    {
        int pointsPerLevel = Mathf.Max(1, classSkillPointsPerLevel + intModifier);
        int totalPoints;

        if (isFirstClassLevel && racialHD == 0)
        {
            // Standard humanoid: 1st level gets x4
            totalPoints = pointsPerLevel * 4;
            if (classLevel > 1)
                totalPoints += pointsPerLevel * (classLevel - 1);
        }
        else
        {
            // Monster adding class levels: no x4 bonus
            totalPoints = pointsPerLevel * classLevel;
        }

        return totalPoints;
    }

    /// <summary>
    /// Get the maximum skill rank for a creature.
    /// </summary>
    public static int GetMaxRank(int totalHD, bool isClassSkill)
    {
        int maxClassRank = totalHD + 3;
        return isClassSkill ? maxClassRank : maxClassRank / 2;
    }

    /// <summary>
    /// Determine if a skill is a class skill for this creature.
    /// Class skills include: the added class's skills + any skills
    /// listed in the creature's original monster entry.
    /// </summary>
    public static bool IsClassSkill(
        string skillName,
        ICharacterClass addedClass,
        HashSet<string> monsterSkills)
    {
        if (addedClass.ClassSkills.Contains(skillName))
            return true;
        if (monsterSkills != null && monsterSkills.Contains(skillName))
            return true;
        return false;
    }
}

```

8. Feat Progression

8.1 Rules

Feats are determined by **total Hit Dice** (racial + class):

```
Feat Count = 1 + floor((TotalHD - 1) / 3)
```

```
HD 1: 1 feat
HD 2-3: 1 feat
HD 3: 2 feats
HD 4-5: 2 feats
HD 6: 3 feats
...
```

Total HD	Feats	Total HD	Feats
1	1	12	4
2	1	13-15	5
3	2	15	6
4-5	2	16-17	6
6	3	18	7
7-8	3	19-20	7
9	4	21	8
10-11	4	—	—

Important: When adding class levels to a monster, do NOT grant retroactive feats for its existing racial HD — those feats are already included in its stat block. Only grant NEW feats for HD that push past the next threshold.

8.2 Implementation

```
public static class CreatureFeatProgression
{
    /// <summary>
    /// Calculate how many total feats a creature should have at the given total HD.
    /// </summary>
    public static int GetTotalFeatCount(int totalHD)
    {
        if (totalHD < 1) return 0;
        return 1 + (totalHD - 1) / 3;
    }

    /// <summary>
    /// Calculate how many NEW feats to grant when advancing a creature.
    /// </summary>
    public static int GetNewFeatsGranted(int previousTotalHD, int newTotalHD)
    {
        return GetTotalFeatCount(newTotalHD) - GetTotalFeatCount(previousTotalHD);
    }

    /// <summary>
    /// Get the HD thresholds at which new feats are gained.
    /// </summary>
    public static List<int> GetFeatThresholds(int maxHD)
    {
        var thresholds = new List<int>();
        for (int hd = 1; hd <= maxHD; hd += 3)
            thresholds.Add(hd);
        return thresholds;
    }
}
```

8.3 Example: Ogre (4 RHD) + Barbarian 2 = 6 total HD

- At 4 RHD: $1 + (4-1)/3 = 2$ feats (already in stat block)
- At 6 total HD: $1 + (6-1)/3 = 2$ feats → 0 new feats from levels 5-6
- At 7 total HD (if Barb 3): $1 + (7-1)/3 = 3$ feats → 1 new feat

9. Ability Score Increases

9.1 Rules

A creature gains +1 to one ability score at every 4 **total** Hit Dice: 4th, 8th, 12th, 16th, 20th.

Critical rule: A monster's base stat block ALREADY includes ability score increases for its racial HD. When adding class levels, only grant increases for NEW total HD thresholds crossed.

9.2 Implementation

```
public static class AbilityScoreProgression
{
    /// <summary>
    /// Calculate how many ability score increases a creature has earned
    /// at the given total HD.
    /// </summary>
    public static int GetTotalIncreases(int totalHD)
    {
        return totalHD / 4;
    }

    /// <summary>
    /// Calculate NEW ability score increases when advancing from old to new total HD.
    /// </summary>
    public static int GetNewIncreases(int oldTotalHD, int newTotalHD)
    {
        return GetTotalIncreases(newTotalHD) - GetTotalIncreases(oldTotalHD);
    }

    /// <summary>
    /// Get the HD thresholds at which increases occur.
    /// </summary>
    public static List<int> GetIncreaseThresholds(int maxHD)
    {
        var thresholds = new List<int>();
        for (int hd = 4; hd <= maxHD; hd += 4)
            thresholds.Add(hd);
        return thresholds;
    }
}
```

9.3 Example: Ogre (4 RHD) + Fighter 4 = 8 total HD

- At 4 RHD: $4/4 = 1$ increase (already in stat block)
 - At 8 total HD: $8/4 = 2$ increases → 1 NEW increase to apply
-

10. Stat Array Application

10.1 Arrays

```

public static class StatArrays
{
    public static readonly int[] Elite      = { 15, 14, 13, 12, 10, 8 };
    public static readonly int[] Nonelite   = { 13, 12, 11, 10, 9, 8 };
    public static readonly int[] Basic      = { 11, 11, 11, 10, 10, 10 };

    /// <summary>
    /// Apply a stat array to a creature, replacing its base scores
    /// and then adding racial modifiers.
    ///
    /// Arrangement priority by class:
    /// - Martial (Fighter, Barbarian, Warrior, Paladin, Ranger): STR > CON > DEX >
WIS > CHA > INT
    /// - Skill/Stealth (Rogue, Bard, Expert): DEX > INT > CHA > CON > WIS > STR
    /// - Divine Caster (Cleric, Druid, Adept): WIS > CON > STR > CHA > DEX > INT
    /// - Arcane Caster (Wizard): INT > DEX > CON > WIS > CHA > STR
    /// - Arcane Caster (Sorcerer): CHA > DEX > CON > WIS > INT > STR
    /// - Social (Aristocrat): CHA > WIS > INT > DEX > CON > STR
    /// - Laborer (Commoner): STR > CON > WIS > DEX > CHA > INT
    /// </summary>
    public static int[] ArrangeForClass(int[] array, string className)
    {
        // Returns [STR, DEX, CON, INT, WIS, CHA] from sorted array
        int[] sorted = (int[])array.Clone();
        System.Array.Sort(sorted);
        System.Array.Reverse(sorted); // Highest first

        int[] result = new int[6]; // [STR, DEX, CON, INT, WIS, CHA]
        int[] priority = GetAbilityPriority(className);

        for (int i = 0; i < 6; i++)
            result[priority[i]] = sorted[i];

        return result;
    }

    /// <summary>
    /// Returns ability index priority (0=STR,1=DEX,2=CON,3=INT,4=WIS,5=CHA)
    /// ordered from highest to lowest priority for the given class.
    /// </summary>
    private static int[] GetAbilityPriority(string className)
    {
        switch (className?.ToLowerInvariant())
        {
            case "fighter": case "barbarian": case "warrior": case "paladin": case "ra
nger":
                return new[] { 0, 2, 1, 4, 5, 3 }; // STR CON DEX WIS CHA INT
            case "rogue": case "bard": case "expert":
                return new[] { 1, 3, 5, 2, 4, 0 }; // DEX INT CHA CON WIS STR
            case "cleric": case "druid": case "adept":
                return new[] { 4, 2, 0, 5, 1, 3 }; // WIS CON STR CHA DEX INT
            case "wizard":
                return new[] { 3, 1, 2, 4, 5, 0 }; // INT DEX CON WIS CHA STR
            case "sorcerer":
                return new[] { 5, 1, 2, 4, 3, 0 }; // CHA DEX CON WIS INT STR
            case "aristocrat":
                return new[] { 5, 4, 3, 1, 2, 0 }; // CHA WIS INT DEX CON STR
            case "commoner":
                return new[] { 0, 2, 4, 1, 5, 3 }; // STR CON WIS DEX CHA INT
            case "monk":
                return new[] { 4, 0, 1, 2, 5, 3 }; // WIS STR DEX CON CHA INT
            default:

```

```

        return new[] { 0, 1, 2, 3, 4, 5 }; // Default: STR DEX CON INT WIS CHA
    }
}

/// <summary>
/// Apply racial modifiers to arranged base scores.
/// </summary>
public static int[] ApplyRacialModifiers(int[] baseScores, int[] racialMods)
{
    int[] result = new int[6];
    for (int i = 0; i < 6; i++)
        result[i] = baseScores[i] + (racialMods != null && i <
racialMods.Length ? racialMods[i] : 0);
    return result;
}
}

```

10.2 When to Use Each Array

Scenario	Array	Example
Monster with PC class levels	Elite	Ogre Barbarian 3
Monster with NPC class levels	Nonelite	Ogre Warrior 3
Generic townsfolk, back-ground NPCs	Basic	Commoner 1
Important NPC with NPC class	Nonelite	Town Guard Captain (Warrior 5)
Important NPC with PC class	Elite	Villain (Fighter 10)
Retain original monster stats	Custom	Any creature kept as-is

11. CR Calculation Formulas

11.1 Associated vs. Nonassociated Classes

Associated class: Synergizes with the monster's existing role. Each level adds **+1 CR**.

Nonassociated class: Doesn't synergize. NPC classes are **always** nonassociated. The CR increase follows a two-stage formula:

```

If nonassociated_levels <= RacialHitDice:
    CR increase = nonassociated_levels / 2 (round down)

If nonassociated_levels > RacialHitDice:
    CR increase = RacialHitDice / 2 (for first RHD levels)
                + (nonassociated_levels - RacialHitDice) (for remaining levels, at +1
each)

```

11.2 Associated Class Guidelines by Creature

Creature	Associated Classes	Reasoning
Ogre	Fighter, Barbarian	Physical combatant
Troll	Fighter, Barbarian, Ranger	Melee + regeneration
Lizardfolk	Druid, Barbarian, Fighter	Tribal warriors
Kobold	Sorcerer, Rogue	Trap-setters, draconic heritage
Gnoll	Ranger, Fighter, Barbarian	Pack hunters
Bugbear	Rogue, Fighter	Stealthy brute
Minotaur	Barbarian, Fighter	Rage + maze navigation
Mind Flayer	Wizard, Sorcerer	Psionic/arcane
Drow	Wizard, Cleric, Fighter, Rogue	Versatile
Dragon	Sorcerer	Innate spellcasting

11.3 Implementation

```

public static class CRCalculator
{
    /// <summary>
    /// Calculate new CR after adding class levels to a creature.
    /// Returns float for fractional CRs (e.g., 0.5 for CR 1/2).
    /// </summary>
    public static float Calculate(
        float baseCR,
        string addedClassName,
        int addedClassLevels,
        int racialHD,
        string[] associatedClasses)
    {
        bool isAssociated = IsAssociated(addedClassName, associatedClasses);

        if (isAssociated)
        {
            // Associated: +1 CR per level
            return baseCR + addedClassLevels;
        }
        else
        {
            // Nonassociated: +1/2 per level up to RHD, then +1 per level
            float crIncrease;
            if (addedClassLevels <= racialHD)
            {
                crIncrease = addedClassLevels / 2f;
            }
            else
            {
                float firstPart = racialHD / 2f;
                float secondPart = addedClassLevels - racialHD;
                crIncrease = firstPart + secondPart;
            }

            return baseCR + crIncrease;
        }
    }

    /// <summary>
    /// Calculate CR for a creature with multiple class stacks.
    /// </summary>
    public static float CalculateMulticlass(
        float baseCR,
        List<ClassLevelEntry> classLevels,
        int racialHD,
        string[] associatedClasses)
    {
        float totalCR = baseCR;
        int totalNonassociatedLevels = 0;

        for (int i = 0; i < classLevels.Count; i++)
        {
            string cls = classLevels[i].ClassName;
            int lvl = classLevels[i].Level;

            if (IsAssociated(cls, associatedClasses))
            {
                totalCR += lvl;
            }
            else
            {

```

```

        totalNonassociatedLevels += lvl;
    }
}

// Apply nonassociated formula to accumulated levels
if (totalNonassociatedLevels > 0)
{
    if (totalNonassociatedLevels <= racialHD)
        totalCR += totalNonassociatedLevels / 2f;
    else
        totalCR += (racialHD / 2f) + (totalNonassociatedLevels - racialHD);
}

return totalCR;
}

/// <summary>
/// Check if a class is associated with the creature.
/// NPC classes are ALWAYS nonassociated.
/// </summary>
public static bool IsAssociated(string className, string[] associatedClasses)
{
    // NPC classes are always nonassociated
    if (ClassRegistryUtils.IsNPCClass(className))
        return false;

    if (associatedClasses == null || associatedClasses.Length == 0)
        return false;

    for (int i = 0; i < associatedClasses.Length; i++)
    {
        if (string.Equals(associatedClasses[i], className, StringComparison.OrdinalIgnoreCase))
            return true;
    }

    return false;
}

/// <summary>
/// Convert float CR to display string (e.g., 0.5 → "1/2", 0.333 → "1/3").
/// </summary>
public static string FormatCR(float cr)
{
    if (cr < 0.4f) return "1/3";
    if (cr < 0.6f) return "1/2";
    return Mathf.RoundToInt(cr).ToString();
}

/// <summary>
/// Calculate CR for a pure NPC-class humanoid (no racial HD).
/// CR = Character Level - 1 (minimum 1/2).
/// </summary>
public static float CalculateNPCClassCR(int characterLevel)
{
    if (characterLevel <= 1) return 0.5f;
    return characterLevel - 1;
}
}

```


11.4 Worked Examples

Creature	Base CR	Added Class	Levels	Association	New CR
Ogre (4 RHD)	3	Barbarian	2	Associated	$3 + 2 = \mathbf{5}$
Ogre (4 RHD)	3	Wizard	4	Nonassociated	$3 + 4/2 = \mathbf{5}$
Troll (6 RHD)	5	Wizard	8	Nonassociated	$5 + 6/2 + 2 = \mathbf{10}$
Kobold ($\frac{1}{2}$ RHD)	$\frac{1}{4}$	Sorcerer	4	Associated	$\frac{1}{4} + 4 = \mathbf{4\frac{1}{4}} \approx \mathbf{4}$
Human (0 RHD)	—	Warrior 5	5	NPC class	$5 - 1 = \mathbf{4}$
Lizardfolk (2 RHD)	1	Druid	5	Associated	$1 + 5 = \mathbf{6}$

12. ECL & Level Adjustment

12.1 Formula

$$\text{ECL} = \text{Racial Hit Dice} + \text{Total Class Levels} + \text{Level Adjustment}$$

ECL is used for XP progression and starting wealth when a monster is used as a PC or cohort.

12.2 Common Level Adjustments

Race/Creature	RHD	LA	Base ECL (no class)
Human	0	+0	0
Drow Elf	0	+2	2
Duergar	0	+1	1
Aasimar	0	+1	1
Tiefling	0	+1	1
Bugbear	3	+1	4
Gnoll	2	+1	3
Lizardfolk	2	+1	3
Ogre	4	+2	6
Minotaur	6	+2	8
Mind Flayer	8	+7	15
Troll	6	+5	11

12.3 Implementation

```

public static class ECLTracker
{
    public static int Calculate(int racialHD, List<ClassLevelEntry> classLevels, int levelAdjustment)
    {
        int totalClassLevels = 0;
        for (int i = 0; i < classLevels.Count; i++)
            totalClassLevels += classLevels[i].Level;
        return racialHD + totalClassLevels + levelAdjustment;
    }

    /// <summary>
    /// Get the wealth budget for a creature based on its ECL.
    /// Uses the Character Wealth by Level table.
    /// </summary>
    public static int GetWealthByECL(int ecl)
    {
        return WealthByLevel.GetWealth(ecl);
    }
}

```

13. Equipment Assignment

13.1 Wealth by Level Table

```
public static class WealthByLevel
{
    private static readonly int[] _wealth = {
        0,          // Level 0 (unused)
        900,         // Level 1
        2700,        // Level 2
        5400,        // Level 3
        9000,        // Level 4
        13000,       // Level 5
        19000,       // Level 6
        27000,       // Level 7
        36000,       // Level 8
        49000,       // Level 9
        66000,       // Level 10
        66000,       // Level 11
        88000,       // Level 12
        110000,      // Level 13
        150000,      // Level 14
        200000,      // Level 15
        260000,      // Level 16
        340000,      // Level 17
        440000,      // Level 18
        580000,      // Level 19
        760000       // Level 20
    };

    public static int GetWealth(int level)
    {
        int clamped = Mathf.Clamp(level, 1, 20);
        return _wealth[clamped];
    }
}
```

13.2 Equipment Budget Allocation

Standard allocation by budget percentage:

Category	Budget %	Example
Primary Weapon	25-30%	+2 Greatsword
Armor	20-25%	+2 Full Plate
Defensive Items	15-20%	Ring of Protection, Amulet of Natural Armor
Ability Enhancement	15-20%	Belt of Giant Strength, Headband of Intellect
Utility/Consumables	10-15%	Cloak of Resistance, potions, scrolls

13.3 Auto-Equip Algorithm

```

public static class NPCEquipmentAssigner
{
    /// <summary>
    /// Auto-equip an NPC based on their class, level, and wealth budget.
    /// </summary>
    public static void Equip(CharacterStats stats, InventoryComponent inv)
    {
        int budget = WealthByLevel.GetWealth(stats.EffectiveCharacterLevel);
        string primaryClass = stats.CharacterClass;
        int remaining = budget;

        // 1. Primary weapon (25-30% of budget)
        int weaponBudget = (int)(budget * 0.28f);
        remaining -= AssignWeapon(stats, inv, primaryClass, weaponBudget);

        // 2. Armor (20-25% of budget)
        int armorBudget = (int)(budget * 0.23f);
        remaining -= AssignArmor(stats, inv, primaryClass, armorBudget);

        // 3. Primary ability enhancement (15-20%)
        int abilityBudget = (int)(budget * 0.18f);
        remaining -= AssignAbilityItem(stats, inv, primaryClass, abilityBudget);

        // 4. Defensive items (15-20%)
        int defenseBudget = (int)(budget * 0.18f);
        remaining -= AssignDefensiveItems(stats, inv, defenseBudget);

        // 5. Utility items with remaining budget
        AssignUtilityItems(stats, inv, remaining);
    }

    /// <summary>
    /// Determine the enhancement bonus affordable within a budget.
    /// Enhancement pricing: +1=2000, +2=8000, +3=18000, +4=32000, +5=50000 (weapons/
    armor base)
    /// </summary>
    public static int GetMaxEnhancementBonus(int budget, bool isWeapon)
    {
        int baseCost = isWeapon ? 2000 : 1000; // Weapon +1 = 2000+300, Armor +1 =
        1000+150
        for (int bonus = 5; bonus >= 1; bonus--)
        {
            int cost = bonus * bonus * baseCost;
            if (cost <= budget) return bonus;
        }
        return 0;
    }
}

```

14. ClassLevelApplier — Core Engine

14.1 Main Entry Point

```

/// <summary>
/// Core engine for applying class levels to any creature.
/// Takes a base NPCDefinition and produces a fully-calculated CharacterStats.
/// </summary>
public static class ClassLevelApplier
{
    /// <summary>
    /// Apply class levels to a base creature, returning a complete CharacterStats.
    /// </summary>
    /// <param name="baseCreature">The base monster/NPC definition.</param>
    /// <param name="className">Class to add (e.g., "Barbarian").</param>
    /// <param name="classLevel">Number of levels to add.</param>
    /// <param name="arrayType">Stat array to apply (Elite for PC classes, Nonelite
for NPC).</param>
    /// <returns>Fully calculated CharacterStats ready for combat.</returns>
    public static CharacterStats Apply(
        NPCDefinition baseCreature,
        string className,
        int classLevel,
        StatArrayType arrayType = StatArrayType.Elite)
    {
        // 1. Create base CharacterStats from creature definition
        CharacterStats stats = CreateBaseStats(baseCreature);

        // 2. Apply stat array (replaces base scores, adds racial mods)
        if (arrayType != StatArrayType.Custom)
        {
            int[] array = GetArray(arrayType);
            int[] arranged = StatArrays.ArrangeForClass(array, className);
            int[] final = StatArrays.ApplyRacialModifiers(arranged, baseCreature.RacialAbilityModifiers);
            ApplyAbilityScores(stats, final);
        }

        // 3. Add class levels
        stats.ClassLevels.Add(new ClassLevelEntry(className, classLevel));
        stats.CharacterClass = className;

        // 4. Calculate ability score increases from new total HD
        int oldTotalHD = baseCreature.RacialHitDice;
        int newTotalHD = stats.TotalHitDice;
        int newIncreases = AbilityScoreProgression.GetNewIncreases(oldTotalHD, newTotalHD);
        ApplyAbilityIncreases(stats, className, newIncreases);

        // 5. Recalculate derived stats
        RecalculateDerivedStats(stats, baseCreature);

        // 6. Allocate skill points
        ICharacterClass classDef = ClassRegistry.GetClass(className);
        if (classDef != null)
        {
            int skillPoints = CreatureSkillAllocator.CalculateClassSkillPoints(
                classDef.SkillPointsPerLevel, classLevel,
                stats.GetModifier(stats.Intelligence), false, baseCreature.RacialHitDice);
            stats.classSkillPointPools[className] = skillPoints;
        }

        // 7. Grant new feats
        int newFeats = CreatureFeatProgression.GetNewFeatsGranted(oldTotalHD, newTotalHD);

```

```

// Feats would be auto-selected or queued for manual selection

// 8. Calculate CR
float newCR = CRCalculator.Calculate(
    ParseCR(baseCreature.ChallengeRating),
    className, classLevel,
    baseCreature.RacialHitDice,
    baseCreature.AssociatedClasses);
stats.ChallengeRating = CRCalculator.FormatCR(newCR);

// 9. Update display name
stats.CharacterName = $"{baseCreature.Name} {className} {classLevel}";

return stats;
}

/// <summary>
/// Recalculate BAB, saves, and HP from racial + class components.
/// </summary>
private static void RecalculateDerivedStats(CharacterStats stats, NPCDefinition baseCreature)
{
    CreatureTypeProgression ctp = CreatureTypeProgressionData-
base.GetFromString(baseCreature.CreatureType);

    // BAB
    int racialBAB = baseCreature.RacialHitDice > 0
        ? ProgressionCalculator.CalculateBAB(ctp.BAB, baseCreature.RacialHitDice)
        : 0;

    int classBAB = 0;
    for (int i = 0; i < stats.ClassLevels.Count; i++)
    {
        ICharacterClass cls = ClassRegistry-
istry.GetClass(stats.ClassLevels[i].ClassName);
        if (cls == null) continue;
        BABProgression prog = BABCalculator.GetBABProgression(cls);
        classBAB += ProgressionCalculator.CalculateBAB(prog, stats.ClassLevels[i].
Level);
    }

    // Store combined BAB (the existing system reads this from CharacterStats)
    stats.Level = stats.TotalHitDice;

    // HP
    int conMod = (stats.Constitution - 10) / 2;
    stats.MaxHP = HPCalculator.Calculate(
        baseCreature.RacialHitDice,
        ctp.HitDie,
        stats.ClassLevels,
        conMod);
}

private static int[] GetArray(StatArrayType type)
{
    switch (type)
    {
        case StatArrayType.Elite: return StatArrays.Elite;
        case StatArrayType.Nonelite: return StatArrays.Nonelite;
        case StatArrayType.Basic: return StatArrays.Basic;
        default: return StatArrays.Elite;
    }
}

```

```
}
}
```

15. Code Structure & Class Hierarchy

15.1 Complete Class Hierarchy

```
ICharacterClass (interface)
├── FighterClass          (existing)
├── RogueClass            (existing)
├── MonkClass             (existing)
├── BarbarianClass        (existing)
├── WizardClass           (existing)
├── ClericClass           (existing)
├── SorcererClass         (existing)
├── RangerClass           (existing)
├── PaladinClass          (existing)
├── BardClass             (existing)
├── DruidClass            (existing)
├── AdeptClass            (NEW – NPC)
├── AristocratClass       (NEW – NPC)
├── CommonerClass         (NEW – NPC)
├── ExpertClass           (NEW – NPC)
├── WarriorClass          (NEW – NPC)

ClassLevelApplier (static – core engine)
├── uses → ClassRegistry
├── uses → CreatureTypeProgressionDatabase
├── uses → CRCalculator
├── uses → ECLTracker
├── uses → HPCalculator
├── uses → BABCalculator
├── uses → SaveCalculator
├── uses → StatArrays
├── uses → AbilityScoreProgression
├── uses → CreatureFeatProgression
├── uses → CreatureSkillAllocator
├── uses → NPCEquipmentAssigner

NPCTemplateDatabase (static – quick generation)
├── uses → ClassLevelApplier
├── uses → NPCDatabase
├── uses → EquipmentByLevel
```


15.2 Data Flow for “Create Ogre Barbarian 3”

1. NPCDatabase.Get("ogre")
 - ➡ NPCDefinition [RacialHD=4, CR="3", CreatureType="Giant", AssociatedClasses=["Fighter", "Barbarian"], RacialAbilityModifiers=[+10, -2, +4, -4, 0, -4], LA=2]
2. ClassLevelApplier.Apply(ogreDef, "Barbarian", 3, StatArrayType.Elite)
3. StatArrayApplier:
 - Elite array [15,14,13,12,10,8] arranged for Barbarian:
 - STR=15, CON=14, DEX=13, WIS=12, CHA=10, INT=8
 - + racial mods: STR=25, DEX=11, CON=18, INT=4, WIS=12, CHA=6
4. ClassLevels = [ClassLevelEntry("Barbarian", 3)]
 - TotalHD = 4 (racial) + 3 (class) = 7
5. AbilityScoreProgression:
 - Old thresholds crossed: 4/4 = 1 (already in racial stats)
 - New thresholds: 7/4 = 1 ➡ 0 new increases
6. BAB:
 - Racial: Giant 4 HD @ Medium = floor(4*3/4) = 3
 - Barbarian 3 @ Good = 3
 - Total BAB = 6
7. Saves:
 - Fort: Giant 4 HD Good = 4, Barb 3 Good = 3 ➡ 7
 - Ref: Giant 4 HD Poor = 1, Barb 3 Poor = 1 ➡ 2
 - Will: Giant 4 HD Poor = 1, Barb 3 Poor = 1 ➡ 2
8. HP:
 - Racial: 1st d8 max=8+4=12, 3xavg(4)+4=24 ➡ 36
 - Barbarian: 3xavg(d12=6)+4=30 ➡ 30
 - Total HP = 66
9. Feats:
 - At 4 RHD: 2 feats (existing)
 - At 7 total HD: 1+(7-1)/3 = 3 ➡ 1 new feat
10. CR:
 - Barbarian is associated: CR = 3 + 3 = 6
11. ECL = 4 (RHD) + 3 (Barb) + 2 (LA) = 9
12. Equipment:
 - Wealth budget = WealthByLevel(9) = 49,000 gp
 - ➡ Assign appropriate Large-sized weapons, armor, items

15.3 File Dependencies

```
ClassLevelApplier.cs
├─ depends on: ClassRegistry.cs (existing)
├─ depends on: CreatureTypeProgression.cs (existing)
├─ depends on: CharacterStats.cs (existing + new fields)
├─ depends on: NPCDatabase.cs (existing + new fields)
├─ depends on: CRCalculator.cs (NEW)
├─ depends on: ECLTracker.cs (NEW)
├─ depends on: HPCalculator.cs (NEW)
├─ depends on: BABCalculator.cs (NEW)
├─ depends on: SaveCalculator.cs (NEW)
├─ depends on: StatArrays.cs (NEW)
├─ depends on: AbilityScoreProgression.cs (NEW)
├─ depends on: CreatureFeatProgression.cs (NEW)
├─ depends on: CreatureSkillAllocator.cs (NEW)
└─ depends on: NPCEquipmentAssigner.cs (NEW)
```

End of Creature Class Application System Design